Estructuras implementadas

Arbol B+ (RF11)

- Implementado con un NodoBMas con una flag esHoja , la cual determina justamente si es una hoja. Los nodos medios o intermedios tienen claves separadoras y punteros a hijos, mientras que las hojas almacenan claves, punteros a envios y un puntero al siguiente.
- Tamaño: orden 4, cada nodo tiene hasta 3 claves. Los arreglos soportan tamano 4 solamente para admitir temporalmente al hacerse la insercion para luego dividirse.
- Búsqueda: Desciende desde la raiz hasta la hoja correspondiente al limite inferior, y luego recorre los punteros siguiente hasta encontrar el deseado. O(log n)

BitSet (RF12)

- Se implemento sobre un unsigned int, que permitiria usar hasta 32 bits (aunque solo usemos 5 en este caso). Cada estado corresponde a un bit, siendo 1 si el envio ya paso por el.
- Operaciones bitwise: Al hacerse sobre un unsigned int, se pueden hacer operaciones por bit aplicando mascaras de corrimiento (and, or, xor)
- Validez historica: Se integra por valor en Envio, lo que provoca que cada vez que se haga una transicion (Envio::cambiarEstado) esta se quede plasmada en el bitset.

Set de códigos (RF13)

- Implementado como una lista simplemente enlazada ORDENADA LEXICOGRAFICAMENTE sin elementos duplicados.
- inserción y unicidad: Recorre la lista buscando la posición ordenada. Si el código ya está, devuelve false y no lo agrega (evita duplicados). Si encuentra un código mayor, frena e inserta el nuevo nodo en ese lugar enlazando los punteros.
- Operaciones entre conjuntos (union, Con, intersection, difference): Como las dos listas ya vienen ordenadas, no hace falta buscar cada elemento en el otro conjunto. Se recorren ambas listas en paralelo con dos punteros (igual que el *merge* de MergeSort) en una sola pasada lineal O(n+m)
- Integración e independencia de datos: El Set solo guarda los códigos, nunca los objetos Envio. Cuando el usuario quiere ver los datos completos de los paquetes resultantes, el sistema busca esos códigos en el Árbol B+. De esta forma no se duplican los paquetes en memoria.

2. Gestión de Memoria y Ownership

- **Dueño de los Envíos:** La clase `ListaDeEnvios` (en `CentroDeDistribucion`) es la única propietaria de los objetos `Envio` creados en el heap, como era previamente.

- **Referencias no propietarias:** El árbol B+ almacena punteros crudos no propietarios. Su destructor libera únicamente las estructuras `NodoBMas`, sin tocar los `Envio`.

- **Propiedad en el Set:** `SetCodigos` gestiona su propia memoria dinámica para los nodos `NodoSet`. Para evitar problemas de doble liberación al retornar por valor en operaciones, se implementó **copia profunda** (constructor de copia y operador de asignación).

- **Ciclo de vida en CentroDeDistribucion:** Las estructuras auxiliares (`indiceBMas`, `mesaClasificacion`, `indice`) se declaran después de `registro`, asegurando que al destruirse el centro se liberen primero los indices y jamás queden punteros colgantes (*dangling pointers*).

- **BitSet:** Se almacena por valor en el stack/objeto, sin consumo dinámico de memoria.

3. Tabla de Complejidades

Tabla de Complejidades Big O — HubFlow

Estructura	Operación	Mejor Caso	Peor Caso	Justificación
B+	Inserción	O(1)	O(log n)	Árbol vacío O(1). En general recorre la altura h = O(log4 n); ante división, la propagación hacia la raíz también está acotada por h.
B+	Búsqueda por código	O(1)	O(log n)	En árbol de 1 nodo es O(1). En general desciende por los nodos internos hasta la hoja: h niveles, es decir O(log4 n).
B+	Búsqueda por rango	O(log n)	O(log n + k)	Baja a la primera hoja en O(log n) y avanza secuencialmente por k envíos mediante el puntero siguiente. Si abarca todo el árbol, k = n.
BitSet	set / reset / test	O(1)	O(1)	Operaciones aritméticas y lógicas de bits directas sobre la máscara (<<, &, |, ~).
BitSet	count	O(1)	O(1)	Recorre únicamente los bits activos hasta el más significativo (a lo sumo 5 iteraciones en nuestro dominio; acotado a 32 bits).
BitSet	AND / OR / XOR	O(1)	O(1)	Operación nativa de una sola instrucción de CPU sobre el entero bits.
Set	Insertar	O(1)	O(n)	Mejor caso: inserción en cabeza o clave menor a la primera. Peor caso: recorrer toda la lista ordenada para insertar al final.

Estructura	Operación	Mejor Caso	Peor Caso	Justificación
B+	Inserción	O(1)	O(log n)	Árbol vacío O(1). En general recorre la altura h = O(log4 n); ante división, la propagación hacia la raíz también está acotada por h.
Set	Eliminar	O(1)	O(n)	Mejor caso: elemento en el primer nodo. Peor caso: elemento en la cola o inexistente.
Set	Buscar pertenencia	O(1)	O(n)	Mejor caso: primer nodo. Peor caso: último nodo o valor mayor a todos.
Set	Unión	O(n + m)	O(n + m)	Recorrido simultáneo de dos listas ordenadas estilo merge en una sola pasada.
Set	Intersección	O(min(n, m))	O(n + m)	Recorrido simultáneo; corta tempranamente si una lista se agota o los rangos son disjuntos.
Set	Diferencia	O(1)	O(n + m)	Mejor caso: primer conjunto vacío. Peor caso: recorrer todos los nodos de A comparando con B.

---

4. Evidencia de Casos de Prueba

Las pruebas automatizadas de la ampliación se encuentran en `tests/test_main.cpp` e integran la totalidad de las funcionalidades solicitadas:

Resultados de los tests:

```
Ampliacion Caso 1 - Arbol B+
  [OK]   Insercion en arbol vacio: se crea una hoja raiz (altura 1)
  [OK]   Con 3 envios la hoja todavia tiene lugar (no se divide)
  [OK]   El 4to envio divide la hoja y se crea una raiz nueva (altura 2)
  [OK]   La clave de separacion de la raiz es PKG-1003 (primera de la hoja derecha)
  [OK]   Se indexaron los 20 envios
  [OK]   Hubo divisiones de nodos internos
  [OK]   Se volvio a crear una raiz nueva y el arbol crecio a 3 niveles o mas
  [OK]   Todas las hojas estan al mismo nivel
  [OK]   Recorriendo las hojas enlazadas los codigos salen ordenados
  [OK]   Cada busqueda devuelve el mismo Envio y visita tantos nodos como niveles tiene el arbol
  [OK]   Al destruir el arbol los Envio siguen vivos (referencias no propietarias)

Ampliacion Caso 2 - Busqueda por rango
  [OK]   Rango PKG-1003..PKG-1006: aparecen los 4 codigos en orden ascendente
  [OK]   No aparecen codigos fuera del rango
  [OK]   Rango que cruza varias hojas: PKG-1005..PKG-1012 devuelve exactamente 8 en orden
  [OK]   El rango baja una sola vez y sigue por las hojas (no recorre todo el arbol)

Ampliacion Caso 3 - BitSet
  [OK]   Recien registrado: solo RECIBIDO (00001)
  [OK]   RECIBIDO, CLASIFICADO y EN_REPARTO -> 00111 (ejemplo de la consigna)
  [OK]   Despues de reprogramar: bit 3 en 1, bit 4 en 0 y count() = 4
  [OK]   Guarda la historia: estuvo REPROGRAMADO aunque ahora este EN_REPARTO
  [OK]   La consulta muestra la cantidad de estados y si estuvo REPROGRAMADO
  [OK]   set/test/count/any sobre 00111
  [OK]   AND: 00111 & 01101 = 00101
  [OK]   OR:  00111 | 01101 = 01111
  [OK]   XOR: 00111 ^ 01101 = 01010
  [OK]   reset(1) apaga solo ese bit

Ampliacion Caso 4 - Set
  [OK]   A = {PKG-1002, PKG-1005, PKG-1008} (queda ordenado)
  [OK]   A U B = {PKG-1002, PKG-1005, PKG-1006, PKG-1008}
  [OK]   A n B = {PKG-1002}
  [OK]   A - B = {PKG-1005, PKG-1008}
  [OK]   La union no repite el elemento compartido
  [OK]   No se admiten duplicados (insert devuelve false)
  [OK]   contains funciona
  [OK]   remove elimina el elemento
  [OK]   Conjunto por zona NORTE
  [OK]   Conjunto por servicio EXPRESS
  [OK]   NORTE n EXPRESS = {PKG-1002}
```

```
Ampliacion Caso 5 - Casos limite
  [OK]   Arbol B+ vacio: altura 0, la busqueda y el rango no encuentran nada
  [OK]   Busqueda de un codigo inexistente
  [OK]   Insercion duplicada en el sistema: se rechaza y el B+ sigue con 6
  [OK]   Insercion duplicada directa en el B+: insertar devuelve false
  [OK]   Busqueda por rango sin resultados
  [OK]   BitSet sin bits activos: any() false y count() 0
  [OK]   Posicion de bit invalida: se rechaza sin modificar el BitSet
  [OK]   Set vacio
  [OK]   Union con un conjunto vacio da el mismo conjunto
  [OK]   Interseccion de conjuntos disjuntos es vacia
  [OK]   Eliminar un elemento inexistente devuelve false
  [OK]   Copiar un Set no comparte nodos con el original

Ampliacion Caso 6 - Integracion
  [OK]   Se registra un envio nuevo
  [OK]   Se lo encuentra con el arbol B+ y se muestran sus datos y los nodos visitados
  [OK]   Su BitSet se actualiza solo al cambiar de estado (RECIBIDO + EN_REPARTO)
  [OK]   Comparar BitSets: estados en comun y estados de uno solo
  [OK]   El envio nuevo aparece en NORTE n EXPRESS
  [OK]   Con los codigos del Set se recuperan los datos completos usando el B+
  [OK]   El B+ apunta al mismo objeto Envio (no se duplico)
```